QA REPORT

1. Introduction

This report presents a quality assurance (QA) analysis of the game across multiple development milestones. It evaluates the evolution of the codebase in terms of structure, maintainability, testing practices, and build performance. The report also reflects on the QA methodology used throughout development, including a change in conceptualisation from Milestone 4 onwards.

The focus of this evaluation is on code quality, code quantity, and build performance, rather than runtime or user-facing performance metrics, due to the scope and nature of the project.

2. QA Methodology

The QA approach combines constructive and analytical techniques.

Constructive QA was applied throughout development to ensure that code adhered to Google coding standards, was properly documented, and included appropriate exception handling. Code quality was maintained through continuous refactoring and developer coordination, particularly to avoid merge conflicts during parallel development. Communication was primarily handled via WhatsApp, supported by regular meetings after each milestone to resolve integration issues and align development progress.

Analytical QA involved evaluation of the system through testing and measurement. This included unit testing, playtesting, and, in later stages, regression testing. Debugging and regression testing were initially treated as corrective QA activities but were later incorporated into analytical QA for simplicity. Automated testing using JUnit was introduced from Milestone 5 onwards.

3. Metrics Collection and Instrumentation

3.1 Data Collection Methods

Metrics were collected using a combination of version control analysis and automated tools. Git was used to retrieve milestone-specific versions of the project by filtering commits using git log --before <date>, where the date corresponded to each milestone deadline. q

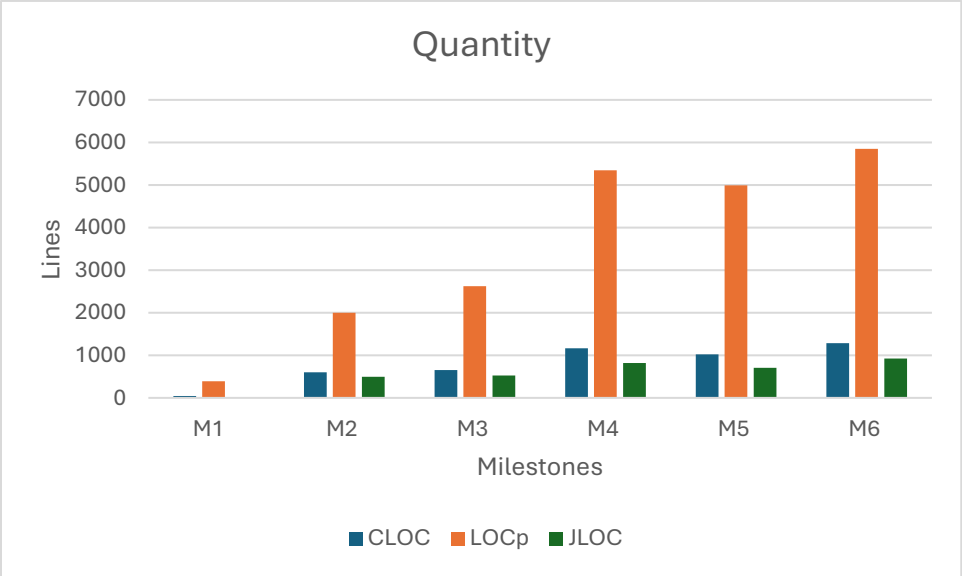
Test coverage was measured using JaCoCo, while code metrics such as complexity and size were obtained using MetricsReloaded. Build performance was measured using Gradle profiling tools with Gradle clean build. All results were recorded manually in Excel for comparison across milestones.

3.2 Scope Limitations

Certain metrics were excluded from analysis. Performance-related metrics such as frame rate, network latency, and server response time were not considered due to the lightweight nature of the game and its operation within a local area network (LAN). Additionally, static analysis warnings and documentation metrics were excluded where they were deemed to be redundant or of low analytical value for this report.

4. Results

4.1 Codebase Size and Documentation

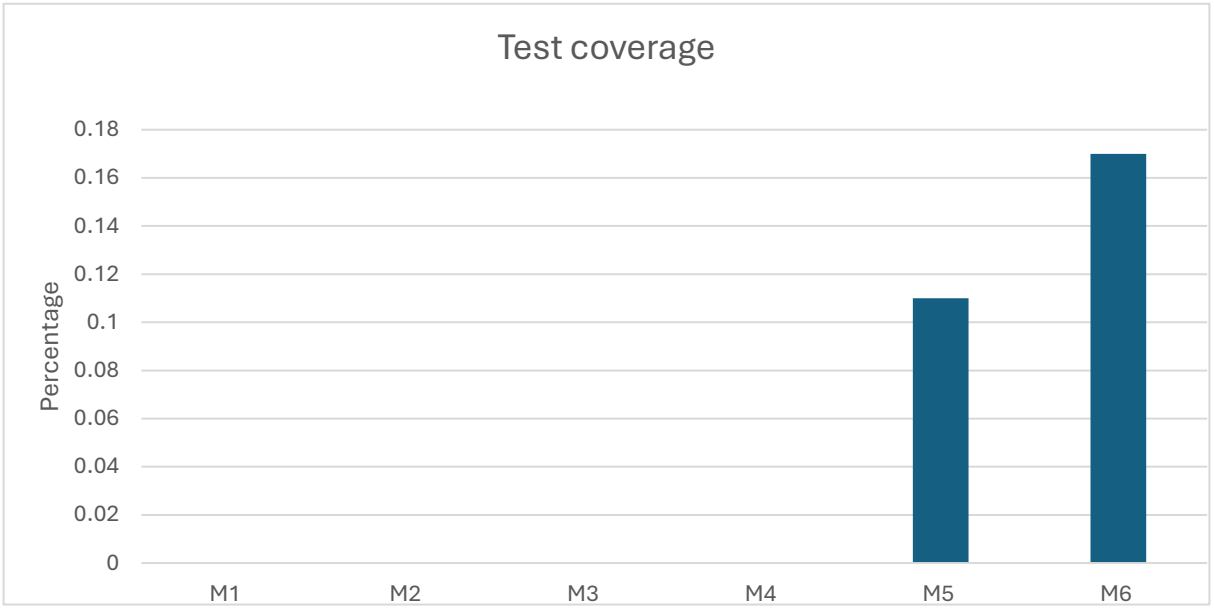


Code size was evaluated using three primary metrics: LOC (lines of code), CLOC (comment lines of code), and JLOC (Javadoc lines of code).

A significant increase in LOC was observed between Milestone 3 and Milestone 4, corresponding to the implementation of core gameplay mechanics and system features. From Milestone 4 to Milestone 5, a slight decrease in LOC occurred due to refactoring efforts that removed redundant code and improved structure.

Despite these changes, documentation levels remained consistently high across all milestones. Both comment density and Javadoc usage were stable, indicating that maintainability and code clarity were consistently prioritised.

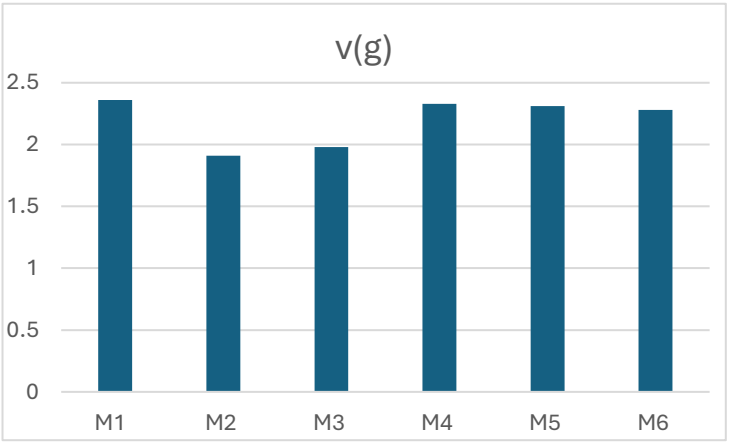
4.2 Unit tests



Unit testing was introduced at Milestone 5, resulting in coverage levels of approximately 12%, increasing to around 18% in Milestone 6. These values exclude trivial classes such as graphical client components.

The late introduction of automated testing limited early validation of system functionality. Initial testing relied primarily on manual playtesting, which provided basic feedback but lacked reproducibility and regression protection. As a result, early milestones had limited automated verification of core functionality, reducing overall testing robustness.

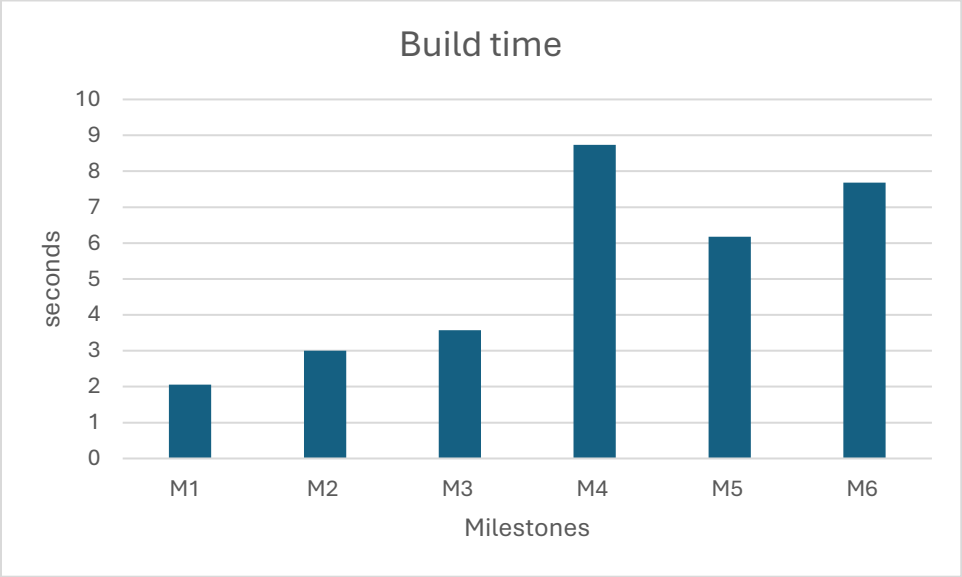
4.3 Complexity



he average cyclomatic complexity remained stable between approximately 2.0 and 2.5 across all milestones. This indicates that methods generally contain a low number of independent execution paths.

This stability suggests that the codebase remained structurally simple despite feature expansion, contributing to readability and maintainability throughout development.

4.4 Build time



Build time was measured using Gradle profiling tools. Across all milestones, build times generally remained below 9 seconds, enabling efficient iteration during development. A noticeable spike occurred at Milestone 4, corresponding to increased codebase size and project complexity.

In later milestones, build times decreased due to caching effects, which improved incremental build performance. However, these results reflect cached builds on a single development machine and therefore do not fully represent clean build or continuous integration performance.

5. Discussion

Overall, the codebase demonstrates strong structural quality. Low cyclomatic complexity, consistent documentation, and efficient build performance indicates a maintainable and well-organised system.

However, weaknesses are present in the QA process, particularly in relation to testing. The late introduction of unit testing significantly reduced early-stage verification and limited regression detection capabilities. As a result, much of the initial validation relied on manual playtesting, which is less reliable and not easily reproducible.

Additionally, while the selected metrics provide useful insight into code structure and maintainability, they do not capture full system quality. Runtime behaviour, user

experience, and cross-device performance were not evaluated, limiting the overall scope of the analysis.

6. Limitations

This evaluation is subject to several limitations. All metrics were collected on a single machine, meaning environmental variation was not accounted for. Build performance results reflect cached incremental builds rather than clean builds, which may underestimate actual build times. Furthermore, no cross-device testing was conducted, and runtime performance metrics were excluded due to scope constraints.

7. Conclusion and Outlook

The project demonstrates a maintainable and structurally stable codebase with consistently low complexity and efficient build performance. Documentation practices remained strong throughout development, and refactoring contributed positively to code quality.

However, the QA process was weakened by the late introduction of automated testing, resulting in limited test coverage and reduced early-stage validation. This highlights a key gap in the development approach.

For future projects, QA planning should be introduced earlier and integrated throughout development. This includes defining testing strategies at the start of the project, implementing unit testing from early development stages, and selecting metrics that more comprehensively reflect system quality, including runtime performance where applicable.